

UNIVERSITY OF PERADENIYA

Department of Computer Engineering

Lessons Learnt from the Second Year Project

Software Systems Design Project and suggestions for future improvements.

E/23/292

K.S. Rambukkanage

My Experience Working on the Project:

As part of the second-year project, I gained practical experience in designing and developing a software system intended to address a real world problem. Our project, the Orthodontics Workflow Automation System, was developed to support the daily operations of an orthodontic clinic. Within this larger system, my focus was on designing the analytics and reporting architecture figuring out how to take raw, everyday clinic activities and transform them into meaningful insights.

At the beginning, I viewed this task simply as extracting information and putting it on a screen. However, after meeting stakeholders and receiving feedback during demonstrations, my perspective shifted entirely. I realized that designing a system for real users requires a deep understanding of their goals. Clinic administrators and instructors do not just want to see data; they need to make quick, critical decisions about patient care, inventory, and clinic efficiency. The information must be accurate, logically structured, and highly secure. This realization taught me to design with empathy, prioritizing the user's context over pure functionality.

One major lesson I learnt was the importance of conceptualizing the data flow before building anything. In a clinical environment, information is heavily interconnected a single patient visit involves appointments, clinical notes, and inventory usage. Designing the analytics meant I had to think abstractly about how to weave these different threads together. I had to figure out how to group and filter massive amounts of information across different timeframes without slowing down the entire system. This taught me to think critically about system efficiency and how structural decisions impact overall performance.

I also learnt how crucial user experience and cognitive psychology are in system design. Presenting data is essentially about communication. If a chart or a report is too cluttered, it fails its purpose. I had to think carefully about how to visually represent complex trends so that a user could understand the clinic's status at a glance. It was not just about making the interface look

good, but about reducing the mental effort required for the staff to interpret the information and take action.

Another critical insight was realizing that system design is a collaborative, constantly evolving agreement. Because the reporting features rely on data generated by every other part of the system, any structural change made by my teammates directly impacted my work. This taught me the importance of maintaining clear boundaries, agreeing on how information should be structured globally, and communicating constantly. A successful system is held together as much by team coordination as it is by logic.

Testing this logic fundamentally changed how I view problem solving. In analytics, a system failure does not always look like an error message; sometimes, it looks like a quietly miscalculated total. I had to train myself to think defensively, anticipating edge cases like missing information, unusual user behavior, or empty date ranges. Designing the system to gracefully handle these unpredictable real-world scenarios was a massive learning curve.

Finally, I learnt that explaining our design choices is just as important as the design itself. A system must survive long after its original creators leave. Documenting our thought process why we structured the data a certain way and how different components interact ensures that future maintainers can understand and build upon our foundation with confidence.

Recommendations

For Staff and Instructors

Staff members should establish strict, standardized procedures for how information is entered into the system during daily operations. The insights generated by the analytics are only as reliable as the data put into it. Furthermore, it is important to carefully manage user roles. Because reports contain sensitive operational and patient data, administrators should ensure that access to these broader insights is restricted strictly to authorized personnel to maintain privacy and security. Regular reviews of system health and data accuracy will also help the system remain reliable over time.

For Future Batches

Future teams should spend time deeply understanding the existing logical structures and how different parts of the system interact before they attempt to add new features. Modifying how data is categorized in one module can drastically alter the accuracy of the reports.

The next team could improve the project by exploring predictive design thinking about how the system might forecast future clinic needs based on current trends, or designing more interactive ways for supervisors to track long term patient progress. Most importantly, future batches must treat documentation as a living part of the design process. Keeping the blueprints and user guides aligned with how the system actually works is the only way to ensure the software remains a sustainable, valuable tool for the clinic.